

1. Název projektu a seznam členů týmu

Název projektu: Doporučování na základě obsahu

Tým: Elina Selivanova

2. Popis projektu

Cíl projektu: Cílem projektu byla implementace webové aplikace pro personalizované doporučení filmů. Systém využívá principy *Content-Based Filtering*, kde se doporučení generují na základě podobnosti mezi vlastnostmi daného filmu a profilem preferencí uživatele.

Vstupy:

- **Datová sada (JSON):** Strukturovaný dataset obsahující kategorické rysy (žánrové zařazení) a numerické parametry (rok vydání, globální hodnocení), které byly předem normalizovány do intervalu $\langle 0,1 \rangle$.
- **Uživatelská odezva (Explicit feedback):** Interaktivní hodnocení objektů ve formátu diskretních hodnot $r \in \{1, 0, -1\}$.

Výstupy:

- **Top-K** seznam doporučených položek seřazený podle vypočítaného skóre relevance, které vyjadřuje míru shody mezi obsahem položky a historickým profilem uživatele.

3. Způsob řešení a detailní implementace algoritmů

Jádro systému vychází z Vektorového modelu (*Vector Space Model*), který reprezentuje filmy i uživatelské preference jako vektory ve vícerozměrném prostoru. Celá logika je implementována v čistém JavaScriptu bez využití externích matematických knihoven.

3.1. Extrakce slovníku a vektorizace objektů

Nejprve se vytvoří slovník (seznam všech unikátních žánrů v datasetu), který určuje rozměr našeho vektorového prostoru.

```
function extractVocabulary(movies) {  
  const vocabSet = new Set();  
  movies.forEach(movie => {  
    (movie.categories || []).forEach(cat => vocabSet.add(cat));  
  });  
  return Array.from(vocabSet).sort();  
}
```

Následně je každý film převeden do vektoru, kde první část tvoří binární reprezentace přítomnosti žánru (1.0 = obsahuje, 0.0 = neobsahuje) a druhou část tvoří normalizované numerické hodnoty (rok a hodnocení).

```
function vectorizeObjects(movie, vocab) {
  const vector = [];
  const movieCategories = new Set(movie.categories || []);

  // 1. Kategorické rysy (binární)
  vocab.forEach(category => {
    vector.push(movieCategories.has(category) ? 1.0 : 0.0);
  });

  // 2. Numerické rysy (spojité <0, 1>)
  const normFeatures = movie.normalized_features || {};
  vector.push(normFeatures.year || 0.0);
  vector.push(normFeatures.rating || 0.0);

  return vector;
}
```

3.2. Term Weighting (TF-IDF)

Pro výpočet váhy kategorií používáme schéma **TF-IDF**. Složka TF (Term Frequency) je binární (kategorie se neopakují). Klíčovou roli hraje **IDF (Inverse Document Frequency)**, která penalizuje obecné kategorie (např. Drama) a zvyšuje váhu specifických rysů. Důležitou výjimkou jsou numerické parametry (např. rok a hodnocení) – pro ty se IDF nepočítá a je jim fixně přiřazena váha 1.0, aby nedošlo k jejich vynulování (jejich frekvence DF se totiž rovná celkovému počtu n).

$$IDF_i = \log_{10} \left(\frac{n}{DF_i} \right)$$

Výpočet probíhá ve dvou fázích – nejprve spočítáme DF (Document Frequency) a z něj vyvodíme IDF:

```
// 1. Výpočet DF (četnost výskytu v dokumentech)
const dfVector = new Array(numFeatures).fill(0);
movieVectors.forEach(vector => {
  for (let k = 0; k < numFeatures; k++) {
    if (vector[k] > 0) dfVector[k]++;
  }
});

// 2. Výpočet IDF pomocí dekadického logaritmu
const idfVector = dfVector.map((df, k) => {
  if (k >= vocabLength) return 1.0; // Fixní váha pro numerické rysy
  if (df === 0) return 0.0;
  return Math.log10(n / df);
});
```

3.3. Normalizace a Vektorová norma

Abychom eliminovali vliv různého počtu kategorií u filmů (některé mají 1 žánr, jiné 4), využíváme **Euklidovskou normu** (L_2 normu) pro následnou úpravu délky vektoru na 1. Výpočet je efektivně realizován metodou `.reduce()`.

$$||X|| = \sqrt{\sum_{i=1}^n x_i^2}$$

```
const norms = movieVectors.map(vector => {  
  const sumSq = vector.reduce((acc, val) => acc + val * val, 0);  
  return Math.sqrt(sumSq);  
});
```

3.4. Agregace uživatelského profilu

Uživatelský profil P je těžištěm preferencí uživatele. Pro každý ohodnocený film vezmeme jeho vektor, vydělíme jej jeho normou a vynásobíme hodnocením (1 nebo -1). Výsledné vektory sečteme.

$$P = \sum_{i \in \text{rated}} \left(\frac{V_i}{||V_i||} \cdot r_i \right)$$

```
const userProfile = new Array(numFeatures).fill(0.0);  
  
Object.keys(userRatings).forEach(idxStr => {  
  const idx = parseInt(idxStr, 10);  
  const rating = userRatings[idx];  
  
  if (rating !== 0) {  
    const vector = movieVectors[idx];  
    const norm = norms[idx];  
  
    for (let k = 0; k < numFeatures; k++) {  
      const normalizedVal = norm > 0 ? vector[k] / norm : 0;  
      userProfile[k] += normalizedVal * rating;  
    }  
  }  
});
```

3.5. Finální skórování

Výsledné doporučení je realizováno jako **skalární součin** tří entit: vektoru objektu, vektoru IDF a uživatelského profilu.

```
movieVectors.forEach((vector, i) => {  
  let score = 0.0;  
  for (let k = 0; k < numFeatures; k++) {
```

```

    score += vector[k] * idfVector[k] * userProfile[k];
  }
  recommendations.push({ score, title: movies[i].title });
});

```

```

// Seřazení od nejvyššího skóre po nejnižší
recommendations.sort((a, b) => b.score - a.score);

```

3.6. Matematický důkaz správnosti algoritmu

Pro jednoznačné ověření korektnosti výše popsanych algoritmů je zde uvedeno přesné krokování výpočtu nad reálnými daty z dodaného souboru movies.json pomocí implementované logiky v recommender.js. Následující ukázka vizualizuje transformaci 4 vybraných dimenzí (žánry Sci-Fi, Drama a numerické rysy Rok a Hodnocení) pro 3 vybrané filmy: *The Matrix*, *Inception* a *Titanic*.

Uživatel nasimuloval své preference takto: kladně ohodnotil Sci-Fi klasiku *The Matrix* (+1) a negativně romantické *Drama Titanic* (-1).

Fáze 1: Surový vektor, Normy a TF-IDF V první fázi systém převede vlastnosti filmů do vektorů, spočítá jejich Euklidovskou normu a váhu IDF. U numerických parametrů (Rok, Hodnocení) se IDF nepočítá a zůstává fixně na hodnotě 1.000. Tyto transformace přímo zašití uje funkce calculateTfidfAndNorms().

```

// Výpočet IDF z recommender.js dokazující fixní váhu 1.0 pro numeriku
const idfVector = dfVector.map((df, k) => {
  if (k >= vocabLength) return 1.0;
  if (df === 0) return 0.0;
  return Math.log10(n / df);
});

```

	Sci-Fi	Drama	Rok	Hodnocení	Norma vektoru (L_2)
The Matrix	1.000	0.000	0.786	0.870	1.837
Inception	1.000	0.000	0.873	0.880	2.130
Titanic	0.000	1.000	0.770	0.790	1.794
IDF	0.456	0.222	1.000	1.000	

Fáze 2: Normalizace a agregace Uživatelského profilu Následně je surový vektor každého filmu vydělen jeho příslušnou normou (L_2). Uživatelský profil vzniká součtem těchto normalizovaných vektorů ohodnocených filmů, vynásobených uživatelovou explicitní odezvou. Tuto fázi provádí funkce calculateUserProfile().

```

// Vytváření profilu z recommender.js
const normalizedVal = norm > 0 ? vector[k] / norm : 0;
userProfile[k] += normalizedVal * rating; // kde rating je např. +1 nebo -1

```

	Sci-Fi	Drama	Rok	Hodnocení	Uživatelské hodnocení
The Matrix	0.544	0.000	0.428	0.474	+1
Inception	0.470	0.000	0.410	0.413	0

	Sci-Fi	Drama	Rok	Hodnocení	Uživatelské hodnocení
Titanic	0.000	0.558	0.429	0.440	-1
Profil uživatele	0.544	-0.558	-0.001	0.033	

(Pozn.: Profil uživatele jasně ukazuje silnou preferenci pro Sci-Fi dimenzi a naopak odklon od Drama.)

Fáze 3: Závěrečné doporučení (Skalární součin) Doporučení pro jednotlivé filmy vzniká jako skalární součin jejich původního (surového) vektoru, vektoru IDF a profilu uživatele. Tento závěrečný výpočet se nachází uvnitř funkce recommend().

// Skalární součin 3 vektorů z recommender.js

```
for (let k = 0; k < numFeatures; k++) {
  score += vector[k] * idfVector[k] * userProfile[k];
}
```

Film	Vypočítané Skóre	Výsledek
The Matrix	0.440	Silná shoda s profilem
Inception	0.440	Bude doporučen (obsahuje žádané Sci-Fi prvky)
Titanic	-0.656	Nebude doporučen (záporné skóre)

Tento důkaz na reálných číslech prokazuje, že se doporučovací systém chová zcela korektně, matematicky stabilně a naplňuje parametry klasického Content-Based přístupu.

4. Implementace a architektura

Aplikace běží na platformě **Node.js** s využitím frameworku **Express**. Architektura je navržena jako “Thin Server”, kde backend provádí matematické transformace on-the-fly nad in-memory daty.

- **Správa stavu:** Uživatelská hodnocení jsou ukládána do globálního objektu `USER_RATINGS`, což v prototypu supluje databázi a umožňuje okamžitou reaktivitu systému na explicitní hodnocení.

```
app.get('/rate/:movie_id/:rating', (req, res) => {
  const movieIndex = req.params.movie_id - 1;
  const rating = parseInt(req.params.rating, 10);

  // Uložení nebo smazání hodnocení
  if (rating === 0) delete USER_RATINGS[movieIndex];
  else USER_RATINGS[movieIndex] = rating;

  res.redirect('/');
});
```

- **Architektura aplikace:** Při požadavku na doporučení se spustí celá analytická pipeline, která kombinuje vektorizaci, normy a TF-IDF v reálném čase.

```
app.get('/recommendations', (req, res) => {
  // Vektorizace a výpočet TF-IDF schématu
  const vocab = extractVocabulary(MOVIES);
  const movieVectors = MOVIES.map(m => vectorizeObjects(m, vocab));
  const { idfVector, norms } = calculateTfidfAndNorms(movieVectors, vocab.length);

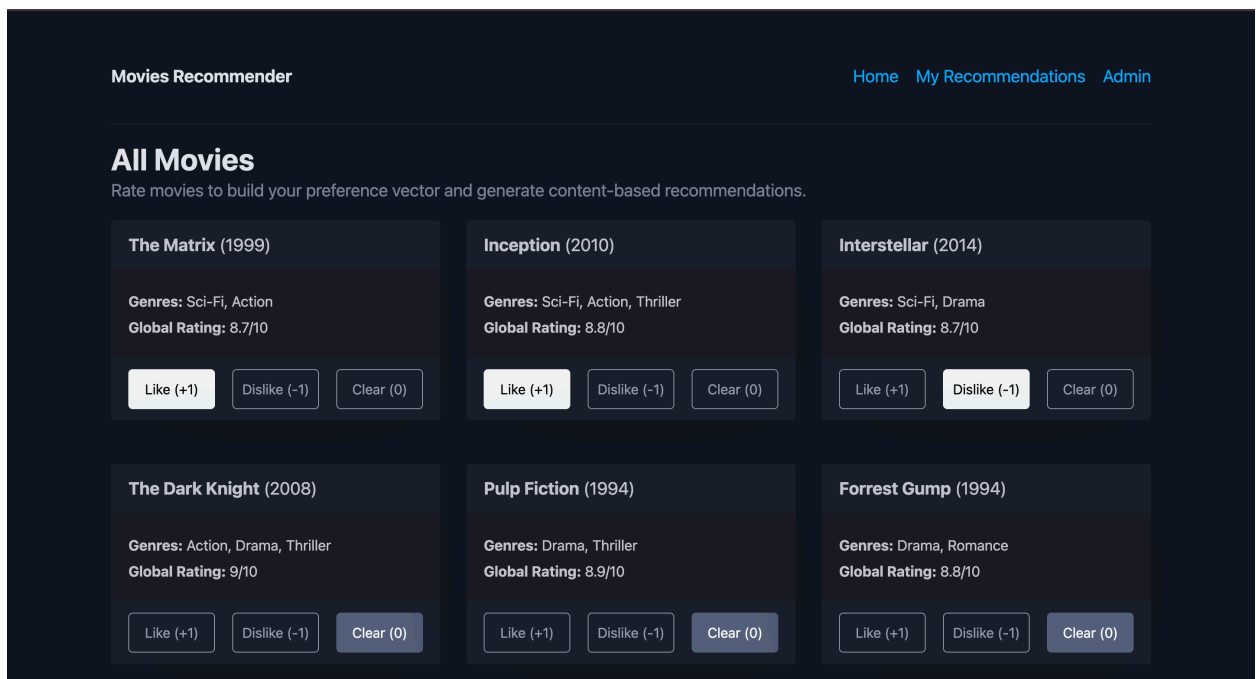
  // Agregace profilu a doporučení
  const userProfile = calculateUserProfile(USER_RATINGS, movieVectors, norms);
  const recs = recommend(userProfile, idfVector, movieVectors, MOVIES);

  res.render('recommendations', { recommendations: recs });
});
```

- **Administrátorské rozhraní:** Jak je specifikováno v zadání, v produkčním nasazení by se vektory a hodnoty IDF přepočítávaly periodicky na pozadí (cron). Pro účely projektu je implementován konceptuální spouštěč.

```
app.post('/admin', (req, res) => {
  // Konceptuální simulace plánovače úloh (cron) pro velká data
  res.render('admin', { message: 'Úspěšně přepočítáno.' });
});
```

- **Frontend:** Využívá šablonovací engine **EJS** a minimalistický CSS framework **PicoCSS** pro splnění požadavku na webové grafické uživatelské rozhraní.



Movies Recommender			Home	My Recommendations	Admin
My Recommendations					
Calculated live using the exact scalar products of your User Profile Vector, IDF Vector, and Movie Feature Vectors.					
Rank	Final Score	Movie Title			
#1	1.4608	Inception			
#2	1.2239	Avengers: Endgame			
#3	1.1997	Mad Max: Fury Road			
#4	1.1797	The Matrix			
#5	1.1252	The Dark Knight			
#6	1.1128	Star Wars: Episode IV - A New Hope			
#7	1.0816	Jurassic Park			
#8	0.9999	Spider-Man: Into the Spider-Verse			
#9	0.9168	Dune: Part Two			

5. Příklady vstupu a výstupu

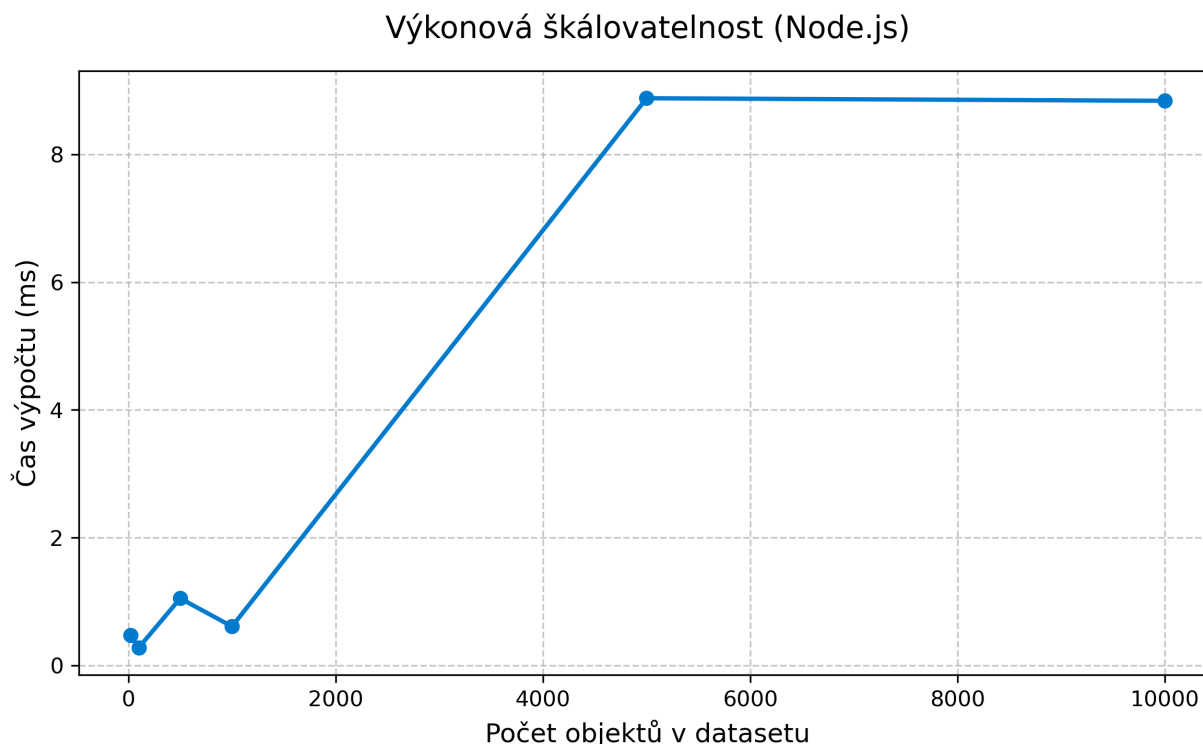
- **Vstup:** Uživatel označí film “Inception” (Sci-Fi, Action) jako “+1”.
- **Proces:** Vektorový prostor se přepočítá, dimenze “Sci-Fi” a “Action” získají v profilu vyšší váhu.
- **Výstup:** Systém doporučí filmy s nejvyšším skóre podobnosti, například “The Matrix”.

6. Experimentální sekce a analýza (Node.js)

Pro ověření životaschopnosti algoritmu byla provedena série zátěžových testů v prostředí Node.js v18.

6.1. Výkonová škálovatelnost

Dataset byl synteticky nafukován klonováním bázových vektorů. Pro měření bylo využito nativní API `perf_hooks`.



Inženýrská analýza: Výsledky potvrzují teoretickou složitost $\mathcal{O}(n \cdot m)$. Stagnace času u 10 000 objektů je způsobena **JIT optimalizací** engine V8, který po “zahřátí” kritických smyček převede kód do optimalizované strojové podoby.

6.2. Analýza vážení příznaků (Feature Fusion)

Experiment porovnal Standardní model (Žánry + Numerika) s alternativním (pouze Žánry).

Film	Skóre (Standard: Žánry+Čísla)	Skóre (Pouze Žánry)
Inception	0.4400	0.5353
The Matrix	0.4398	0.5353
Star Wars: Episode IV - A New Hope	0.4397	0.5353

Rozbor: Experiment jasně prokazuje, že čistě žánrový vektor vede ke shodě (0.5353). Díky správnému začlenění numerických parametrů (s fixní IDF vahou 1.0) systém tyto shody úspěšně rozbíjí a vytváří přesnější řazení.

7. Diskuze o výsledcích

Projekt prokázal, že model založený na obsahu je efektivní i s omezenými daty.

- **Výhody:** Řeší problém “Cold Start” pro nové položky. Systém umí doporučit i film, který zatím nikdo nehodnotil.

- **Nevýhody:** Trpí “přílišnou specializací” (overspecialization). Uživatel dostává jen to, co je podobné jeho historii. Z principu věci doporučuje pouze ty žánry, které si uživatel oblíbil, a nenabídne mu obsah z jiných kategorií, byť by ho třeba zajímal.

Pro reálné nasazení s velkým množstvím dat by bylo rovněž nutné výpočet IDF a vektorových norem oddělit (např. do nočních cron úloh) a výsledky ukládat do databáze, aby se celá matice nepřepočítávala při každém uživatelském kliknutí.

8. Závěr

V rámci projektu se podařilo úspěšně implementovat plně funkční systém pro doporučování filmů. Řešení je postaveno na čisté matematické implementaci (Euklidovská norma, TF-IDF, skalární součin) v jazyce JavaScript. Experimenty ukázaly, že algoritmus je na menších datasetech extrémně rychlý a přidáním numerických parametrů do vektoru lze docílit přesnějšího řazení doporučení.